

How AppComponent and UserComponent talk to each other

Who sends, who receives, and how it works, explained with the code in `01-starting-project`.

1 The short version

AppComponent is the parent **PARENT** and **UserComponent is the child** **CHILD**. You can tell because the parent's template contains `<app-user>` and the parent lists `UserComponent` in its `imports` array.

They talk in two directions:

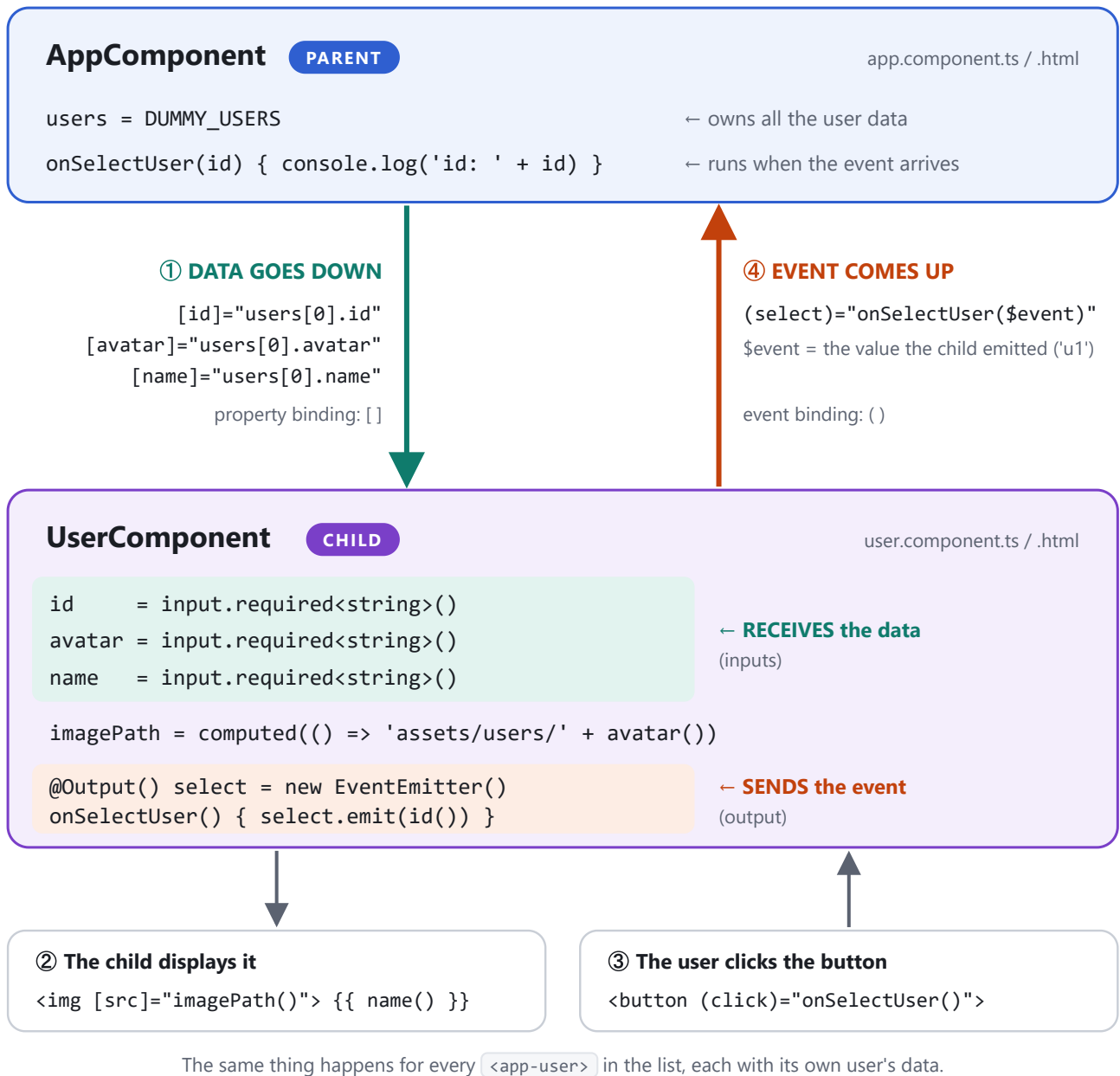
DIRECTION	WHAT TRAVELS	HOW THE PARENT WRITES IT	HOW THE CHILD DECLARES IT
DOWN parent → child	Data (<code>id</code> , <code>avatar</code> , <code>name</code>)	<code>[name]="..."</code> square brackets	<code>name = input.required<string>()</code>
UP child → parent	Events (this user was clicked)	<code>(select)="..."</code> parentheses	<code>@Output() select = new EventEmitter()</code>

An easy way to remember it

`[ ]` sends **data in** to the child `( )` listens for **events coming out** of the child

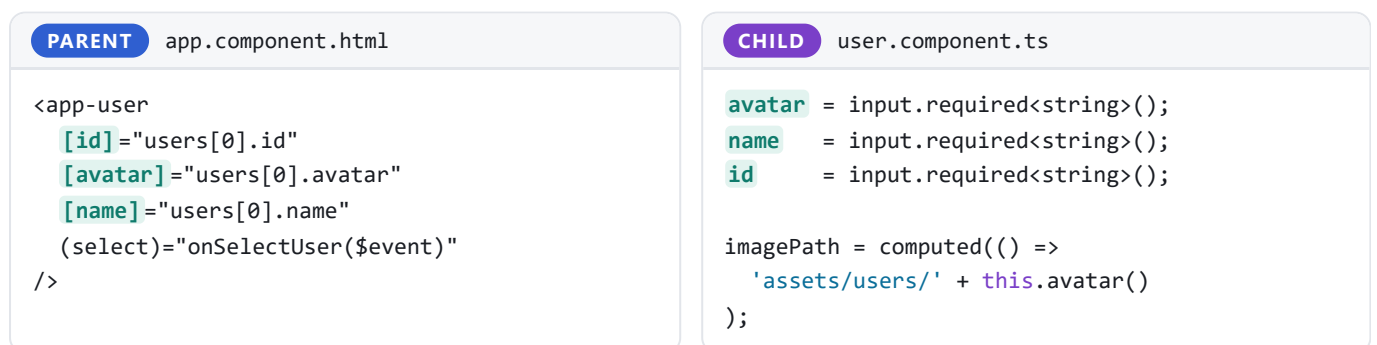
2 The big picture

Follow the numbers ① → ④. Data flows down the green path and the click comes back up the orange path.



3 Part 1: The parent sends data down **DOWN**

The names have to match. **[name]** in the parent fills **name** in the child.



- 1 `AppComponent` has `users = DUMMY_USERS` (*app.component.ts, line 15*). The parent owns the data.
- 2 In *app.component.html* it creates `<app-user>` and passes values in with `[id]`, `[avatar]` and `[name]`.

- 3 `UserComponent` receives them through `input.required<string>()` (*user.component.ts, lines 22–24*). `required` means the parent **must** pass them, or Angular shows an error.
- 4 The child reads each value by **calling the signal**, like `name()` and `imagePath()`, in its template:

```
CHILD user.component.html

<button (click)="onSelectUser()">
  <img [src]="imagePath()" [alt]="name()" />
  <span>{{ name() }}</span>
</button>
```

4 Part 2: The child sends an event up UP

The output's name has to match too. The child's **select** is what the parent listens to with **(select)**.

```
CHILD user.component.ts

@Output() select = new EventEmitter();

onSelectUser() {
  this.select.emit(this.id);
}
```

```
PARENT app.component.html + .ts

<app-user ... (select)="onSelectUser($event)" />

onSelectUser(id: string) {
  console.log('id: ' + id);
}
```

- 1 The user clicks the button. `(click)="onSelectUser()"` runs the **child's** `onSelectUser()`.
- 2 That calls `this.select.emit(...)`, which means "I was selected, and here's a value."
- 3 The parent is listening with `(select)="onSelectUser($event)"`. `$event` holds whatever the child passed to `emit()`.
- 4 The **parent's** `onSelectUser(id)` runs and logs the id to the console.

Two methods with the same name

Both components have a method called `onSelectUser`, but they are **two different methods**. The child's handles the click, and the parent's handles the event. Renaming one (for example `onUserSelected` in the parent) can make this easier to follow.

5 A bug you'll see in the console

In *user.component.ts, line 29*, `id` is a **signal**, so you have to **call it** to get the string value.

X Now: sends the signal itself

```
this.select.emit(this.id);

// console: id: [Input Signal: u1]
```

✓ Fix: sends the value 'u1'

```
this.select.emit(this.id());

// console: id: u1
```

It also helps to give the emitter a type, so TypeScript can catch this kind of mistake for you:

```
@Output() select = new EventEmitter<string>();
```

6 Two smaller things

- Lines 38–40 of *app.component.html* are plain text ("task of the user component is..."), so that sentence **shows up on the page**. If it's meant as a note, wrap it in an HTML comment: `<!-- ... -->`.
- There's a newer, signal-based way to write outputs that matches your `input()` style:
`select = output<string>()`; Emitting works the same way, with `this.select.emit(this.id())`.

Remember

- The component whose template **uses** the other one (`<app-user>`) is the **parent**.
- **Parent → child:** `[input]="value"` in the parent, `input()` in the child.
- **Child → parent:** `.emit(value)` in the child, `(output)="handler($event)"` in the parent.
- Signals are read by calling them: `id()`, not `id`.